SecureDT System Architecture

Intelligent and Secure Task Scheduling for Digital Twin-Enabled Edge-Cloud Networks

Version 1.0

1. Frontend
Jinja2 HTML templates rendered by Django backend. ~35 template files covering all UI pages (login, register, dashboard, GA, digital twin, attack simulation, storage attack, analytics, experiments, etc.). Static assets: CSS (style.css), JavaScript (app.js for UI interactions and API polling), fonts, and images. All client-side logic communicates with the backend via Django REST Framework API endpoints. No Groq API key exposed to browser - all Groq calls are backend-only.

2. Django Backend
Django 5.2.16 provides the web framework, authentication, admin interface, and REST API via Django REST Framework. URL routing in securedt/urls.py. Core logic in SecureDT/app.py (Flask-style route handlers adapted for Django) and engine modules. Database abstraction via Django ORM (MySQL in production, SQLite in development). Middleware: Security, Session, CSRF, Auth, Messages. Security headers automatically set (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy). Session management with SECRET_KEY. All secrets (GROQ_API_KEY, SECRET_KEY) loaded from .env environment variable never hardcoded in source.

3. MySQL Database
Primary relational database for persistent data: user accounts, task schedules, attack logs, experiment results, Digital Twin state, analytics data, and model metrics. Django ORM models for: User (extended auth), Task, AttackEvent, ResourceState, Storage, Experiment, AI Prediction, Groq Status. Tables include indexes on resource_type, task_status, attack_type, and experiment_status for performance. Chunking and streaming for large dataset handling (100K-10M+ rows). MySQL connector (PyMySQL) configured via DATABASES setting in settings.py. Example configuration placeholders in .env:
`DB_ENGINE=django.db.backends.mysql DB_NAME=securedt DB_USER=securedt DB_PASSWORD=your_password DB_HOST=localhost DB_PORT=3306`

4. AI Engine
Two-tier AI architecture: Primary - Local scikit-learn Random Forest model (models/risk_model.pkl) for task risk prediction, explanation, and anomaly scoring. Optional enhancement - Groq Cloud LLM insight layer (when GROQ_API_KEY configured) provides natural-language explanations, experiment summaries, and risk commentary. Core security decisions remain deterministic and application-controlled regardless of Groq availability. AI features: risk level prediction (safe/trained/critical), confidence scores, feature importance, deterministic fallback when model file missing, JSON-serializable metrics via /api/ai/analyze. Training script: scripts/train_ai.py on ai_training_10000.csv.

5. Genetic Algorithm
GA scheduler in engine/genetic_algorithm.py and engine/scheduler.py. Population: 40 chromosomes (task-to-resource assignments). Fitness function weighted sum: latency (0.25), energy (0.15), privacy (0.20), security (0.30), deadline (0.10). Tournament selection (size 3), single-point crossover (rate 0.85), per-gene mutation (rate 0.12), elitism (top 4 chromosomes). Generations: 50. Output: best chromosome (optimal resource assignment), fitness history. Baseline comparators: random, greedy, privacy-aware schedules. All optimization runs are deterministic with seed 42 (config.DATASET_SEED).

6. Digital Twin
Virtual resource state tracker in engine/digital_twin.py. Tracks CPU, memory, latency, energy, active tasks, availability, trust, and security-risk for each virtual resource (LOCAL, MEC-1/2/3, CLOUD). Real/simulated state synchronization with the scheduler. Risk level property (SAFE/WARNING/HIGH/CRITICAL/QUARANTINED) updates dynamically. Attack events update resource risk scores and may trigger quarantine. Resource states stored in MySQL (digital_twin_resourcestate table) and in-memory for real-time access via /api/state. Three-tier topology: edge (LOCAL), MEC (MEC-1/2/3), cloud (CLOUD) with base nominal characteristics defined in config.py RESOURCES dict.

7. Security
Layered security architecture: Authentication (Django built-in auth, session-based, detailed in /api/auth/login/register). Authorization (role-based access, superuser vs. regular user). CSRF protection on all

state-changing API requests. XSS prevention via CSP and output encoding. SQL injection defense via Django ORM parameterized queries. File upload validation (CSV only, 8MB max, 100K row max, re-encoded on upload). Rate limiting (per-IP, default 60 requests/minute). Security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS). Audit logging (all events recorded with category, severity, export to CSV). Groq key protection: key never exposed in browser, read-only from .env, exception handling, timeout (30s), response validation, logging without secrets. Attack sandbox: all simulations confined to digital twin and virtual stores; no real network attacks, filesystem commands, or database operations.

8. Dataset Engine

Six synthetic datasets (seed 42): tasks_10000.csv, attack_events_5000.csv, resource_states_3000.csv, experiment_results_2000.csv, ai_training_10000.csv, storage_security_5000.csv. One-click big-data generation up to 10M+ rows via scripts/import_real_datasets.py with chunking and streaming. Dataset Manager page (/datasets) provides registry, validation, normalization to UnifiedEvent schema, quality reports (missing values, duplicates, invalid numerics, 0-100 score + grade), and JSON/CSV export. Big-data features: chunked CSV writing, Parquet/DuckDB conversion when pyarrow/duckdb installed, pagination for UI, background processing via Celery (optional).

9. Celery & Redis (Optional)

Optional background task queue for large dataset processing, experiment campaigns, and long-running GA optimizations. Celery with Redis as broker offloads tasks from the web worker. Fallback: all operations work synchronously in Django development mode when Celery/Redis unavailable. Not required for core functionality.

10. Groq Cloud LLM

Optional enhancement layer for natural-language explanations, experiment summaries, and risk commentary. Backend-only integration: Browser <---> Django Backend <---> Groq API. Key stored exclusively in .env environment variable - never in frontend, source code, database, or Git. If GROQ_API_KEY missing: app shows 'Groq AI is not configured. Local AI mode is active.' and continues fully offline. Groq may be used for: AI explanations, experiment explanations, risk explanations, security-event summaries, natural-language analytics, research/project assistant. Groq must NOT control: attack execution, filesystem commands, network attacks, database deletion, system commands, or security bypasses. Configuration: GROQ_API_KEY, GROQ_MODEL, GROQ_ENABLED, GROQ_TIMEOUT in .env or Django settings. Default model: llama-3.3-70b-versatile.

11. Telemetry (Optional Zeek)

Optional Zeek network security monitoring integration. When ZEEK_LOGS_DIR environment variable configured, /api/telemetry/status reports live telemetry data. When not configured, reports simulation mode with clear message. UI never fails on missing adapter. Provides additional context for security monitoring and attack correlation.

12. API Endpoints

Django REST Framework API with ~50 endpoints. Core categories: state/health APIs, auth (login/register/reset), task management (generate, upload, list, retrieve), scheduling (GA, baselines), attack simulation (start/mitigate/stop), digital twin state, security monitor, storage attack, routing recalibration, analytics, experiments, AI analysis and explanation, Groq status, application security posture check (30+ OWASP checks), audit logs, experiment management. Consistent JSON error envelope - no stack traces leaked to client. Rate limiting on expensive endpoints. CSRF tokens on state-changing requests. All API responses follow standard envelope format.